

Data Structures & Algorithms (DSA TSC)

Solution Document & Problem Explanations

Authors: Shivansh & Shishir

September 7, 2026

Contents

1	Linked Lists Foundations	2
2	Array & Cycle Detection	2
3	Memory-Efficient Data Structures	3
4	Stacks & Queues	4
5	Monotonic Stack Problems	4
6	Complex Queue Variations	5
7	Asymptotics & Trees	6
8	Algorithmic Tree Exercises	6
9	Tree Representations & Analysis	8
10	Heaps & Priority Queues	9
11	Sliding Window Techniques	10

1 Linked Lists Foundations

Problem: Linked Lists Overview

Review the key differences between Singly Linked Lists (SLL) and Doubly Linked Lists (DLL), including structural attributes, traversal directions, and operational constraints.

Solution & Analysis: SLL vs. DLL Comparison

- **Singly Linked List (SLL):**
 - Node contains **data** and a single pointer **next**.
 - Traversal is strictly **unidirectional** (forward only).
 - Insertion/Deletion after a given node is $\mathcal{O}(1)$, but deletion of a given node requires $\mathcal{O}(n)$ time to find its predecessor unless the predecessor pointer is provided.
- **Doubly Linked List (DLL):**
 - Node contains **data**, **next**, and **prev** pointers.
 - Supports **bidirectional** traversal.
 - Allows $\mathcal{O}(1)$ deletion of a node given its pointer directly (since `node->prev->next = node->next`).

Important Constraint

Neither SLL nor DLL permits direct random access (indexing like arrays). Accessing the i -th element requires $\mathcal{O}(i)$ linear traversal.

2 Array & Cycle Detection

Problem: Find the Duplicate Number

Given an array of integers `nums` containing $N + 1$ integers where each integer is in the range $[1, N]$ inclusive. There is **exactly one** repeated number.

Constraints: Solve without modifying `nums` and use only $\mathcal{O}(1)$ extra space.

Solution & Analysis: Floyd's Cycle-Finding Algorithm (Tortoise & Hare)

Because each element `nums[i] ∈ [1, N]`, we can map the array to a functional graph where an edge exists from index i to index `nums[i]`. Since $N + 1$ nodes point to indices within $[1, N]$, by the Pigeonhole Principle, at least two indices point to the same value, forming a **cycle**.

Algorithm Steps

1. **Phase 1 (Detect Cycle):** Advance `slow` by 1 step (`slow = nums[slow]`) and `fast` by 2 steps (`fast = nums[nums[fast]]`) until they intersect.
2. **Phase 2 (Find Cycle Entry Point):** Reset `slow` to `nums[0]` while keeping `fast` at the intersection point. Advance both one step at a time (`slow = nums[slow]`, `fast = nums[fast]`). The point where they meet is the duplicate number!

Mathematical Proof of Correctness

Let F be the distance from the start to the cycle entry, C be the cycle length, and a be the distance from entry to the intersection point.

$$\text{Distance traveled by slow} = F + a$$

$$\text{Distance traveled by fast} = F + a + kC \quad (k \geq 1)$$

Since fast moves twice as fast as slow:

$$2(F + a) = F + a + kC \implies F + a = kC \implies F = kC - a$$

Thus, moving F steps from the start and F steps from the intersection point leads both pointers to the exact cycle entry node.

Complexity: Time Complexity = $\mathcal{O}(N)$, Space Complexity = $\mathcal{O}(1)$.

3 Memory-Efficient Data Structures

Problem: XOR Doubly Linked List

Implement a doubly linked list using only one pointer variable $x.np$ per node instead of two (**next** and **prev**). Describe access requirements for the head and show how to implement **SEARCH**, **INSERT**, and **DELETE**.

Solution & Analysis: Bitwise XOR Pointer Architecture

The pointer field $x.np$ stores the bitwise XOR sum of the memory addresses of the previous and next nodes:

$$x.np = \text{address}(\text{prev}) \oplus \text{address}(\text{next})$$

Head Access Requirement

To traverse or access the list starting at **head**, we **must keep both** the head pointer and a NULL (address 0) pointer. Knowing $\text{prev} = \text{NULL}$, we retrieve next via:

$$\text{next} = x.np \oplus \text{NULL} = x.np$$

Operations Implementation

1. **SEARCH(value):** Start with $\text{curr} = \text{head}$ and $\text{prev} = \text{NULL}$. At each node:

$$\text{next} = \text{prev} \oplus \text{curr}.np$$

Set $\text{prev} = \text{curr}$ and $\text{curr} = \text{next}$ until $\text{curr}.data == \text{value}$ or $\text{curr} == \text{NULL}$.

2. **INSERT_FRONT(newNode):**

$$\text{newNode}.np = \text{NULL} \oplus \text{head}$$

if $\text{head} \neq \text{NULL}$: $\text{head}.np = (\text{old_next_of_head}) \oplus \text{newNode}$ where $\text{old_next} = \text{head}.np \oplus \text{NULL}$
 $\text{head} = \text{newNode}$

3. **DELETE(node):** Let prev and next be the pointers to neighboring nodes obtained during traversal. Update neighbors:

$$\text{prev}.np = (\text{prev}.np \oplus \text{node}) \oplus \text{next}$$

$$\text{next}.np = (\text{next}.np \oplus \text{node}) \oplus \text{prev}$$

4 Stacks & Queues

Problem: Stacks and Queues Fundamentals

Summarize core operations and key algorithmic applications of Stacks (LIFO) and Queues (FIFO).

Solution & Analysis: Operational Overview

- **Stack (LIFO):** Operations `push(x)`, `pop()`, `top()` run in $\mathcal{O}(1)$.
 - *Key Application:* Monotonic Stacks for Next Greater Element, Expression Evaluation, DFS.
- **Queue (FIFO):** Operations `enqueue(x)`, `dequeue()`, `front()` run in $\mathcal{O}(1)$.
 - *Key Application:* Level-order traversal, BFS, Task Buffers.

5 Monotonic Stack Problems

Problem: Largest Rectangle in Histogram

Given an array of heights representing histogram bars of width 1, find the maximum area of a rectangle in the histogram. Example: Input = [2, 1, 5, 6, 2, 3] $\implies$ Output = 10.

Solution & Analysis: Monotonic Stack Approach

We maintain a monotonic increasing stack of indices. An index is popped when we encounter a height smaller than the height at the stack's top index.

Algorithm

1. Iterate through all bars i from 0 to N (appending a dummy bar of height 0 at $i = N$).
2. While stack is non-empty and $\text{heights}[i] < \text{heights}[\text{stack.top}()]$, pop index $h = \text{stack.pop}()$.
3. The height of the rectangle bounded by bar h is $\text{heights}[h]$.
4. The width is calculated as:

$$\text{width} = \begin{cases} i & \text{if stack is empty} \\ i - \text{stack.top}() - 1 & \text{otherwise} \end{cases}$$

5. Calculate area = height $\times$ width and update maximum area.

Complexity: $\mathcal{O}(N)$ Time, $\mathcal{O}(N)$ Space (each element pushed/popped at most once).

Problem: Min/Max Stack

Design a stack supporting `push`, `pop`, `top`, `getMin`, and `getMax` all in strictly $\mathcal{O}(1)$ time complexity.

Solution & Analysis: $O(1)$ Aux Stack / Value Encoding Solution

Method 1: Auxiliary Stacks

Maintain three parallel stacks:

- **mainStack**: stores actual values.
- **minStack**: stores the minimum value up to the current element.
- **maxStack**: stores the maximum value up to the current element.

When pushing x :

```
minStack.push(min(x, minStack.top()))
maxStack.push(max(x, maxStack.top()))
```

Method 2: Value Encoding (Without Extra Space for Min)

When pushing a new minimum $x < \text{current_min}$: Push encoded value $V = 2x - \text{current_min}$ onto the stack, and update $\text{current_min} = x$. When popping an element $V < \text{current_min}$: Restore previous minimum: $\text{prev_min} = 2 \cdot \text{current_min} - V$.

6 Complex Queue Variations

Problem: The Mess Table Queue Problem (Team Queue)

Students from k hostels line up for meals.

- **Enqueue**: If a student sees a peer from their hostel in queue, they cut line to join directly behind the last peer of their hostel. Otherwise, they join at the back.
- **Dequeue**: Standard FIFO processing from the front.

Analyze efficient data structure implementation, time complexities, and queue metrics.

Solution & Analysis: Design and Statistical Analysis

1. Efficient Implementation

Maintain:

1. A **Global Queue** storing hostel IDs currently present in the line.
2. An array/hash map of **Hostel Queues**, where `hostelQueue[i]` holds the students of hostel i .

2. Operations Complexity

- **Enqueue(student, hostel_id)**:
 - If `hostelQueue[hostel_id]` is empty, push `hostel_id` to Global Queue.
 - Push student into `hostelQueue[hostel_id]`.
 - Time Complexity: $O(1)$.
- **Dequeue()**:

- Look at `front_hostel = GlobalQueue.front()`.
- Pop student from `hostelQueue[front_hostel]`.
- If `hostelQueue[front_hostel]` becomes empty, pop `front_hostel` from Global Queue.
- Time Complexity: $\mathcal{O}(1)$.

3. Waiting Time and Variance Comparison

- **Average Waiting Time:** Remains **identical** to a standard FIFO queue overall under steady arrival, as total work/service rate is unchanged.
- **Variance / Fairness Metric:** The ****variance in individual waiting time increases significantly****. Students arriving behind a large hostel block suffer long delays (starvation risk), violating strict FIFO fairness.

7 Asymptotics & Trees

Problem: Definition of Big-O Notation

Formalize Big-O asymptotic notation for functions $f(n)$ and $g(n)$.

Solution & Analysis: Formal Mathematical Definition

Let $f(n), g(n) : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. We write $f(n) = \mathcal{O}(g(n))$ if and only if there exist positive constants $c > 0$ and $n_0 \geq 1$ such that:

$$0 \leq f(n) \leq c \cdot g(n) \quad \forall n \geq n_0$$

Problem: Overview of Trees

Summarize essential graph-theoretic properties and traversals of trees.

Solution & Analysis: Tree Properties

An N -node tree is an undirected, connected, acyclic graph featuring:

- Exactly $N - 1$ edges.
- Unique simple path between any pair of nodes.
- **Traversals:** DFS (Pre-order, In-order, Post-order) and BFS (Level-order).

8 Algorithmic Tree Exercises

Problem: Traffic Signals Exercise

Street length x . Add n traffic signals one by one at positions between 0 and x . After each addition, output the length of the longest road segment without signals.

Solution & Analysis: Data Structure Optimization

1. Naive Solution: $\mathcal{O}(n^2)$

Maintain sorted array of signals. Recompute adjacent gaps on each insertion.

2. Optimized Solution: $\mathcal{O}(n \log n)$

Use a balanced BST (`std::set`) for signal locations, and a `std::multiset` for segment lengths. When signal p is inserted between lower bound L and upper bound R :

1. Remove length $(R - L)$ from lengths multiset.
2. Insert lengths $(p - L)$ and $(R - p)$ into lengths multiset.
3. Insert p into signal set.
4. Max segment length is always `*lengths.rbegin()`.

3. Reverse Deletion (DSU) Solution: $\mathcal{O}(n)$

Process signal insertions backwards as deletions using Disjoint Set Union (DSU) or Doubly Linked Lists.

Problem: Invert Binary Tree

Given the root of a binary tree, invert (mirror) the binary tree.

Solution & Analysis: Recursive Tree Inversion

Swap the left and right pointers for every node recursively:

```
1  TreeNode* invertTree(TreeNode* root) {  
2      if (root == nullptr) return nullptr;  
3      TreeNode* temp = root->left;  
4      root->left = invertTree(root->right);  
5      root->right = invertTree(temp);  
6      return root;  
7  }
```

Complexity: $\mathcal{O}(N)$ Time, $\mathcal{O}(H)$ Auxiliary Space (recursion stack height H).

Problem: Tree Metrics & LCA

In a binary tree T with parent pointers:

1. Find the farthest node from a given node n .
2. Calculate the diameter of T .
3. Find the Lowest Common Ancestor (LCA) of nodes p and q .

Solution & Analysis: Solutions

1. Farthest Node from n

Run BFS/DFS starting from node n treating parent pointers as undirected graph edges. The last node visited is the farthest node. Time: $\mathcal{O}(N)$.

2. Tree Diameter

Run two BFS passes:

1. First BFS from arbitrary node u finds node v farthest from u .
2. Second BFS from v finds node w farthest from v .
3. The distance between v and w is the tree diameter!

3. LCA with Parent Pointers

Calculate depths d_p and d_q . Lift the deeper node up by $|d_p - d_q|$ steps. Then advance both nodes up simultaneously until $p == q$. Time: $\mathcal{O}(H)$, Space: $\mathcal{O}(1)$.

9 Tree Representations & Analysis

Problem: General Trees to Binary Trees

Describe the Left-Child Right-Sibling (LCRS) representation converting general ordered trees to binary trees and vice versa.

Solution & Analysis: LCRS Transformation Rules

For every non-root node i in general tree T :

- $\text{left_child}(b(i)) = b(\text{first_child}(i))$
- $\text{right_child}(b(i)) = b(\text{next_right_sibling}(i))$

Conversion Algorithms

- **General to Binary:** Recursively attach a node's first child as its left binary child, and link subsequent siblings sequentially along right binary pointers.
- **Binary to General:** Left child becomes the first child; following the right chain of that left child yields all siblings of that child in order.

Problem: Average Case Analysis of BST Insertion

Derive the average case path length $T(n)$ for inserting n keys in random permutation into an empty BST.

Solution & Analysis: Mathematical Proof via Recurrence

When n keys are inserted in random order, any key is equally likely to be the root with probability $\frac{1}{n}$. The internal path length $P(n)$ satisfies the recurrence:

$$P(n) = P(k) + P(n - 1 - k) + (n - 1)$$

Taking expectation over all $k \in [0, n - 1]$:

$$E[P(n)] = \frac{2}{n} \sum_{k=0}^{n-1} E[P(k)] + n - 1$$

Multiplying by n :

$$nE[P(n)] = 2 \sum_{k=0}^{n-1} E[P(k)] + n(n-1)$$

Subtracting the equation for $(n-1)$:

$$nE[P(n)] - (n-1)E[P(n-1)] = 2E[P(n-1)] + 2(n-1)$$

$$nE[P(n)] = (n+1)E[P(n-1)] + 2(n-1)$$

Dividing by $n(n+1)$:

$$\frac{E[P(n)]}{n+1} = \frac{E[P(n-1)]}{n} + \frac{2(n-1)}{n(n+1)}$$

Summing the telescope series yields:

$$E[P(n)] = 2(n+1)H_n - 4n \approx 2n \ln n \approx 1.386n \log_2 n = \mathcal{O}(n \log n)$$

The average search/insertion depth is therefore $2 \ln n \approx 1.386 \log_2 n$.

10 Heaps & Priority Queues

Problem: Heap Exercises

1. Efficient algorithm to merge two binary heaps of size M and N .
2. Can a binary tree with > 2 nodes be both a BST and a Heap?
3. Merge k sorted linked lists each containing N elements.

Solution & Analysis: Detailed Answers

1. Merging Two Binary Heaps

Concatenate both array representations into a single array of size $M + N$, then run Floyd's `buildHeap` algorithm in $\mathcal{O}(M + N)$ time.

2. BST and Heap Intersection Problem

Answer: NO (assuming distinct keys).

- BST requires: $\text{left} < \text{root} < \text{right}$.
- Max-Heap requires: $\text{root} > \text{left}$ AND $\text{root} > \text{right}$.
 - Contradiction: BST requires $\text{root} < \text{right}$, but Max-Heap requires $\text{root} > \text{right}$!
- Min-Heap requires: $\text{root} < \text{left}$ AND $\text{root} < \text{right}$.
 - Contradiction: BST requires $\text{root} > \text{left}$, but Min-Heap requires $\text{root} < \text{left}$!

Thus, no valid binary tree with > 2 nodes and distinct keys can simultaneously satisfy both invariants.

3. Merging k Sorted Lists

Use a Min-Heap of size k storing the current head nodes of the k lists.

1. Insert head of each list into heap: $\mathcal{O}(k \log k)$.
2. Extract minimum element from heap, append to merged list, and push next element from that extracted list into heap.

Complexity: Time = $\mathcal{O}(kN \log k)$, Auxiliary Space = $\mathcal{O}(k)$.

11 Sliding Window Techniques

Problem: Sliding Window Maximum

Find the maximum element in each sliding window of size K across an array of size N .

Solution & Analysis: Monotonic Double-Ended Queue (Deque)

Maintain a deque storing indices of array elements in monotonically decreasing order of their corresponding values.

Algorithm Steps

For each index i from 0 to $N - 1$:

1. Remove indices out of current window scope ($< i - K + 1$) from front of deque.
2. Pop indices from back of deque while $\text{nums}[\text{deque.back}()] \leq \text{nums}[i]$.
3. Push index i to back of deque.
4. For $i \geq K - 1$, maximum for window is $\text{nums}[\text{deque.front}()]$.

Complexity: $\mathcal{O}(N)$ Time (each index pushed/popped at most once), $\mathcal{O}(K)$ Space.